

International Cybersecurity and Digital Forensics Academy (ICDFA)**CIP-B103: Network Forensics Fundamentals****LAB 3: SYN Flood Attack Investigation Using tshark****Three-Week Delivery Block 1/3 | 18-19 July 2026**

Prepared by: Aminu Idris, AMCPN | Founder, ICDFA

Important Pre-Lab Instruction Read the matching lecture slide deck before executing the practical. Work only inside the ICDFA-approved lab or your own isolated virtual environment. Record every command, observation and screenshot as contemporaneous evidence.	
Course Code	CIP-B103
Course Title	Network Forensics Fundamentals
Student Name	Fuseini Imoru Kantuogaa
Instructor's Name	Dr. Sarah James
Lab Number	Lab 3
Lab Title	SYN Flood Attack Investigation Using tshark
Delivery Block	1/3 of 3
Scheduled Dates	18-19 July 2026
Estimated Duration	3-4 hours practical work plus report writing
Mode	Individual practical lab in an authorized virtual environment
Required Evidence	mySYNFloodCapture.pcap or a bounded loopback capture generated in the lab

1. Source Materials and Slide Alignment

- HTTP, tshark and SYN Flood: TCP three-way handshake, half-open connections, SYN flood concept, Scapy packet generation and tshark capture.
- The uploaded slide uses Apache on port 80, loopback capture and a four-packet controlled simulation.

- The assignment requires two authorized VMs or an equivalent isolated environment and evidence that distinguishes attack-like SYN activity from a normal handshake.

Reading Requirement Before Lab Execution

The lab deliberately follows the sequence, terminology and core exercises presented in the uploaded CIP-B103 slides. Commands have been corrected where needed for Linux syntax and forensic repeatability.

2. Lab Scenario

An Apache service experienced repeated TCP SYN packets that were not followed by the final ACK of the three-way handshake. You will preserve the packet capture, establish a normal-traffic baseline, identify suspicious SYN patterns, calculate indicators, and conduct a strictly bounded loopback simulation to validate your interpretation.

3. Learning Outcomes

- Differentiate a complete TCP handshake from an incomplete or half-open attempt.
- Use tshark display filters and field extraction to identify SYN, SYN-ACK, ACK and reset packets.
- Calculate SYN counts, unique source ports, response behaviour and incomplete-handshake indicators.
- Create a four-packet loopback-only simulation with Scapy in an authorized environment.
- Develop a defensible timeline and incident finding from PCAP evidence.
- Recommend controls for detecting and mitigating SYN flood activity.

4. Legal, Ethical and Safety Rules

- Use only systems, packet captures and networks for which you have explicit authorization.
- Keep the lab isolated from production, campus, office and public networks. Use NAT or host-only virtual networking as instructed.
- Do not collect, disclose or reuse real credentials, personal messages or confidential traffic.
- Preserve original capture files. Perform analysis on verified working copies and record SHA-256 hashes.
- Stop any traffic-generation or interception process immediately after the required evidence is captured.
- Restore firewall, ARP, forwarding and network settings before closing the lab.
- The simulation is limited to four SYN packets against 127.0.0.1 or an instructor-assigned private lab IP. Do not increase packet count, automate continuous sending, spoof third-party addresses or target public systems.

5. Required Tools and Environment

Item	Recommended Tool	Purpose
Operating system	Kali Linux VM	Run tshark, Apache and Scapy.
Target service	Local Apache on 127.0.0.1:80	Safe target for bounded simulation.
Capture tool	tshark/Wireshark	Capture and analyze TCP flags.
Packet library	python3-scapy	Generate a small number of SYN packets.
Evidence	Provided PCAP or student capture	Forensic analysis source.
Optional second VM	Authorized host-only target	Demonstrate network capture without touching external networks.

6. Lab Folder Structure and Evidence Preparation

```
mkdir -p ~/CIP-B103-Lab3/{evidence,working,exported,reports,screenshots,scripts}
cd ~/CIP-B103-Lab3
pwd
find . -maxdepth 1 -type d -print
```

Create the folder structure before downloading or generating evidence. Store original captures under evidence and analysis copies under working.

```
sudo apt update
sudo apt install -y apache2 tshark wireshark python3-scapy
sudo systemctl enable --now apache2
# Optional supplied capture
wget -O evidence/mySYNFloodCapture.pcap \
'https://raw.githubusercontent.com/frankwxu/digital-forensics-
lab/main/Illegal_Possession_Images/lab_files/SYN_Flood/mySYNFloodCapture.pcap'
sha256sum evidence/mySYNFloodCapture.pcap | tee reports/syn_capture_sha256.txt
```

Screenshot required: The Apache service state, evidence file details and SHA-256 hash.

```
(kali㉿kali)-[~]
└─$ mkdir -p ~/CIP-B103-Lab3/{evidence,working,exported,reports,screenshots,scripts}
cd ~/CIP-B103-Lab3

(kali㉿kali)-[~/CIP-B103-Lab3]
└─$ pwd
/home/kali/CIP-B103-Lab3

(kali㉿kali)-[~/CIP-B103-Lab3]
└─$ find . -maxdepth 1 -type d -print
.
./working
./screenshots
./evidence
./scripts
./reports
./exported

(kali㉿kali)-[~/CIP-B103-Lab3]
└─$ tree ~/CIP-B103-Lab3
/home/kali/CIP-B103-Lab3
├── evidence
├── exported
├── reports
├── screenshots
├── scripts
├── working
└── 7 directories, 0 files

(kali㉿kali)-[~/CIP-B103-Lab3]
└─$
```

Screenshot 1: Evidence folder structure created (mkdir, cd, pwd, find, tree).

```
(kali㉿kali)-[~/CIP-B103-Lab3]
└─$ sudo apt update
[sudo] password for kali:
Hit:1 http://http.kali.org/kali kali-rolling InRelease
1734 packages can be upgraded. Run 'apt list --upgradable' to see them.

(kali㉿kali)-[~/CIP-B103-Lab3]
└─$ sudo apt install -y apache2 tshark wireshark python3-scapy
apache2 is already the newest version (2.4.68-1).
apache2 set to manually installed.
tshark is already the newest version (4.6.6-1).
tshark set to manually installed.
wireshark is already the newest version (4.6.6-1).
wireshark set to manually installed.
python3-scapy is already the newest version (2.7.0+dfsg1-1).
python3-scapy set to manually installed.
The following packages were automatically installed and are no longer required:
  libraw23t64 libvidstab1.1 openjdk-21-jre openjdk-21-jre-headless
Use 'sudo apt autoremove' to remove them.

Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 1734

(kali㉿kali)-[~/CIP-B103-Lab3]
└─$ sudo systemctl enable --now apache2
Synchronizing state of apache2.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable apache2

(kali㉿kali)-[~/CIP-B103-Lab3]
└─$ systemctl status apache2
● apache2.service - The Apache HTTP Server
```

Screenshot 2: Package updates, tool installation, and apache2 enabled.

```
(kali㉿kali)-[~/CIP-B103-Lab3]
└─$ systemctl status apache2
● apache2.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: disabled)
   Active: active (running) since Sun 2026-07-26 09:00:32 EDT; 1h 53min ago
     Invocation: 2fcc79af483740eda369fc8dc55f0709
       Docs: https://httpd.apache.org/docs/2.4/
    Main PID: 949 (apache2)
      Status: "Total requests: 10; Idle/Busy workers 100/0;Requests/sec: 0.00147; Bytes served/sec: 19 B/sec"
        Tasks: 7 (limit: 2108)
       Memory: 20.8M (peak: 37.2M)
          CPU: 1.078s
    CGroup: /system.slice/apache2.service
            └─949 /usr/sbin/apache2 -k start -DFOREGROUND
              └─996 /usr/sbin/apache2 -k start -DFOREGROUND
                └─997 /usr/sbin/apache2 -k start -DFOREGROUND
                  └─998 /usr/sbin/apache2 -k start -DFOREGROUND
                    └─999 /usr/sbin/apache2 -k start -DFOREGROUND
                      └─1000 /usr/sbin/apache2 -k start -DFOREGROUND
                        └─8928 /usr/sbin/apache2 -k start -DFOREGROUND

Jul 26 09:00:32 kali systemd[1]: Starting apache2.service - The Apache HTTP Server ...
Jul 26 09:00:32 kali apachectl[949]: AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 127.0.1>
Jul 26 09:00:32 kali systemd[1]: Started apache2.service - The Apache HTTP Server.
```

Screenshot 3: systemctl status confirming Apache2 active and listening.

```
(kali㉿kali)-[~/CIP-B103-Lab3]
└─$ wget -O evidence/mySYNFloodCapture.pcap \
'https://raw.githubusercontent.com/frankwxu/digital-forensics-lab/main/Illegal_Possession_Images/lab_files/SYN_Flood/mySYNFloodCapture.pcap'

--2026-07-26 10:54:48-- https://raw.githubusercontent.com/frankwxu/digital-forensics-lab/main/Illegal_Possession_Images/lab_files/SYN_Flood/mySYNFloodCapture.pcap
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.108.133, 185.199.109.133, 185.199.110.133, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.108.133|:443 ... connected.
HTTP request sent, awaiting response... 200 OK
Length: 260716 (255K) [application/octet-stream]
Saving to: 'evidence/mySYNFloodCapture.pcap'

evidence/mySYNFloodCapture.pcap 100%[=====>] 254.61K 1.18MB/s in 0.2s

2026-07-26 10:54:49 (1.18 MB/s) - 'evidence/mySYNFloodCapture.pcap' saved [260716/260716]

(kali㉿kali)-[~/CIP-B103-Lab3]
└─$ ls -lh evidence/
total 256K
-rw-rw-r-- 1 kali kali 255K Jul 26 10:54 mySYNFloodCapture.pcap

(kali㉿kali)-[~/CIP-B103-Lab3]
└─$ sha256sum evidence/mySYNFloodCapture.pcap | tee reports/syn_capture_sha256.txt
14765b029a72e9c41dd8b4d32f5b1d2c7d9efee0f084949151f183a39baa55f5 evidence/mySYNFloodCapture.pcap

(kali㉿kali)-[~/CIP-B103-Lab3]
└─$
```

Screenshot 4: mySYNFloodCapture.pcap downloaded, listed, and SHA-256 hashed.

7. Mini Evidence and Chain-of-Custody Worksheet

Field	Student Entry
Case/lab identifier	CIP-B103-Lab3-YourFullName
Trainee name	[Insert trainee full name — not shown in the supplied screenshots; terminal user is 'kali']
Date and time started	26 July 2026, approx. 08:55–09:00 local (Kali VM) — inferred from the Apache2 service start time (09:00:32 EDT) shown in systemctl status
Evidence file name(s)	mySYNFloodCapture.pcap; normal_http.pcapng; bounded_syn_activity.pcapng (working copy: bounded_syn_activity_working.pcapng)

Field	Student Entry
Source or generation method	mySYNFloodCapture.pcap downloaded via wget from the course GitHub repository (supplied reference evidence); normal_http.pcapng and bounded_syn_activity.pcapng captured locally with tshark on loopback (lo, tcp port 80) while Apache2 served requests and a 4-packet Scapy SYN probe (scripts/syn_probe_lab.py) ran
Original SHA-256	mySYNFloodCapture.pcap: 14765b029a72e9c41dd8b4d32f5b1d2c7d9efeedf084949151f183a39baa55f5 — bounded_syn_activity.pcapng: 695f2e51f86fc56e493e09caef1badfc189f42a609204fb38dea63bb1f1ae944
Working-copy SHA-256	bounded_syn_activity_working.pcapng: 695f2e51f86fc56e493e09caef1badfc189f42a609204fb38dea63bb1f1ae944 — identical to original
Analysis workstation/VM	Kali Linux (user: kali), CIP-B103-Lab3 working directory
Notes on any changes	None. cp --preserve=timestamps used for the bounded-activity working copy; SHA-256 matched exactly. normal_http.pcapng was captured directly to evidence/ as root then chown'd to kali:kali (ownership only, no content change).

8. Part A - Establish a Normal Handshake Baseline

```

sudo tshark -i lo -f 'tcp port 80' -a duration:15 -w evidence/normal_http.pcapng &
sleep 2
curl --no-keepalive http://127.0.0.1/ >/dev/null
wait

tshark -r evidence/normal_http.pcapng -Y 'tcp.flags.syn==1 || tcp.flags.fin==1' -T fields \
  -e frame.number -e frame.time -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport -e tcp.flags \
  | tee reports/normal_handshake_flags.tsv

```

Document the expected SYN, SYN-ACK and ACK sequence before examining suspicious traffic.


```

(kali㉿kali)-[~/CIP-B103-Lab3]
$ sudo tshark -i lo -f 'tcp port 80' -a duration:20 -w /tmp/normal_http.pcapng
Running as user "root" and group "root". This could be dangerous.
Capturing on 'Loopback: lo'

File System
(kali㉿kali)-[~/CIP-B103-Lab3]
$ sudo tshark -i lo -f 'tcp port 80' -a duration:20 -w /tmp/normal_
http.pcapng
Running as user "root" and group "root". This could be dangerous.
Capturing on 'Loopback: lo'
10

(kali㉿kali)-[~/CIP-B103-Lab3]
$ sudo cp /tmp/normal_http.pcapng evidence/
sudo chown kali:kali evidence/normal_http.pcapng

(kali㉿kali)-[~/CIP-B103-Lab3]
$ ls -lh evidence/normal_http.pcapng
-rw-r--r-- 1 kali kali 13K Jul 26 11:26 evidence/normal_http.pcapng

(kali㉿kali)-[~/CIP-B103-Lab3]
$ tshark -r evidence/normal_http.pcapng -Y 'tcp.flags.syn==1 || tcp.flags.fin==1'
-T fields \
-e frame.number -e frame.time -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport -e
tcp.flags \
| tee reports/normal_handshake_flags.tsv
1      2026-07-26T11:25:24.030379939-0400      127.0.0.1      40198      127.0.0.1
      80      0x0002
2      2026-07-26T11:25:24.030391815-0400      127.0.0.1      80      127.0.0.1
      40198      0x0012
8      2026-07-26T11:25:24.031857171-0400      127.0.0.1      40198      127.0.0.1
      80      0x0011
9      2026-07-26T11:25:24.031970109-0400      127.0.0.1      80      127.0.0.1
      40198      0x0011

```

Screenshot 5: Baseline tshark capture on loopback, copied/chowned, and SYN/FIN flag extraction.

```

zsh: corrupt history file /home/kali/.zsh_history
(kali㉿kali)-[~/CIP-B103-Lab3]
$ curl --no-keepalive http://127.0.0.1/ >/dev/null
% Total    % Received % Xferd  Average Speed   Time    Time     Current
           %    100    10703   0      6.25M    0      0      0
100  10703 100  10703   0      6.25M    0      0      0

(kali㉿kali)-[~/CIP-B103-Lab3]
$ 

```

Screenshot 6: curl request to the local Apache service generating the normal HTTP session.

9. Part B - Conduct the Bounded Loopback Simulation

Create the following script exactly. It sends four SYN packets to the local Apache service and then exits. It does not run continuously and does not target another organization.

```
cat > scripts/syn_probe_lab.py <<'PYLAB'
from scapy.all import IP, TCP, RandShort, send

TARGET = '127.0.0.1'
PORT = 80
COUNT = 4

packets = [IP(dst=TARGET)/TCP(sport=RandShort(), dport=PORT, flags='S') for _ in range(COUNT)]
send(packets, verbose=False)
print(f'Sent {COUNT} authorized training SYN packets to {TARGET}:{PORT}')
PYLAB

# Terminal 1
sudo tshark -i lo -f 'tcp port 80' -c 20 -w evidence/bounded_syn_activity.pcapng

# Terminal 2, while capture is running
sudo python3 scripts/syn_probe_lab.py
```

Stop Condition

If the capture reaches 20 packets it stops automatically. Otherwise stop with Ctrl+C immediately after the four SYN packets and associated responses are captured.

```
(kali㉿kali)-[~/CIP-B103-Lab3]
$ cd ~/CIP-B103-Lab3

(kali㉿kali)-[~/CIP-B103-Lab3]
$ cat > scripts/syn_probe_lab.py <<'PYLAB'
from scapy.all import IP, TCP, RandShort, send

TARGET = '127.0.0.1'
PORT = 80
COUNT = 4

packets = [IP(dst=TARGET)/TCP(sport=RandShort(), dport=PORT, flags='S') for _ in ra
send(packets, verbose=False)
print(f'Sent {COUNT} authorized training SYN packets to {TARGET}:{PORT}')
PYLAB

(kali㉿kali)-[~/CIP-B103-Lab3]
$ ls -l scripts/
total 4
-rw-rw-r-- 1 kali kali 288 Jul 26 11:30 syn_probe_lab.py
```


Screenshot 7: Bounded 4-packet Scapy SYN probe script created and listed.

```
(kali㉿kali)-[~/CIP-B103-Lab3]
$ sudo tshark -i lo -f 'tcp port 80' -c 20 -w /tmp/bounded_syn_activity.pcapng
Running as user "root" and group "root". This could be dangerous.
Capturing on 'Loopback: lo'
12 ^C

(kali㉿kali)-[~/CIP-B103-Lab3]
$ sudo cp /tmp/bounded_syn_activity.pcapng evidence/

(kali㉿kali)-[~/CIP-B103-Lab3]
$ sudo chown kali:kali evidence/bounded_syn_activity.pcapng

(kali㉿kali)-[~/CIP-B103-Lab3]
$ ls -lh evidence/
total 276K
-rw----- 1 kali kali 1.4K Jul 26 11:34 bounded_syn_activity.pcapng
-rw-rw-r-- 1 kali kali 255K Jul 26 10:54 mySYNFloodCapture.pcap
-rw----- 1 kali kali 13K Jul 26 11:26 normal_http.pcapng

(kali㉿kali)-[~/CIP-B103-Lab3]
$ tshark -r evidence/bounded_syn_activity.pcapng -Y 'tcp.flags.syn==1' -T fields
-e frame.number -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport -e tcp.flags \
| tee reports/bounded_syn_packets.tsv
1      127.0.0.1      51631  127.0.0.1      80      0x0002
2      127.0.0.1      80     127.0.0.1      51631   0x0012
4      127.0.0.1      45323  127.0.0.1      80      0x0002
5      127.0.0.1      80     127.0.0.1      45323   0x0012
7      127.0.0.1      9145   127.0.0.1      80      0x0002
8      127.0.0.1      80     127.0.0.1      9145    0x0012
10     127.0.0.1      33581  127.0.0.1      80      0x0002
11     127.0.0.1      80     127.0.0.1      33581   0x0012

(kali㉿kali)-[~/CIP-B103-Lab3]
$
```

Screenshot 8: Bounded loopback capture started, copied/chowned, and SYN packet list extracted.

```
zsh: corrupt history file /home/kali/.zsh_history
(kali㉿kali)-[~/CIP-B103-Lab3]
$ sudo python3 scripts/syn_probe_lab.py
[sudo] password for kali:
Sent 4 authorized training SYN packets to 127.0.0.1:80

(kali㉿kali)-[~/CIP-B103-Lab3]
$
```

Screenshot 9: syn_probe_lab.py executed, sending 4 authorized training SYN packets.

```

(kali@kali)~/CIP-B103-Lab3
$ ls -lh evidence/bounded_syn_activity.pcapng
-rw-r--r-- 1 kali kali 1.4K Jul 26 11:34 evidence/bounded_syn_activity.pcapng

(kali@kali)~/CIP-B103-Lab3
$ cd ~/CIP-B103-Lab3

(kali@kali)~/CIP-B103-Lab3
$ mkdir -p working

(kali@kali)~/CIP-B103-Lab3
$ cp --preserve=timestamps evidence/bounded_syn_activity.pcapng working/bounded_syn_activity_working.pcapng

(kali@kali)~/CIP-B103-Lab3
$ ls -lh working/
total 4.0K
-rw-r--r-- 1 kali kali 1.4K Jul 26 11:34 bounded_syn_activity_working.pcapng

(kali@kali)~/CIP-B103-Lab3
$ sha256sum evidence/bounded_syn_activity.pcapng working/bounded_syn_activity_working.pcapng | tee reports/bounded_capture_hashes.txt
695f2e51f86fc56e493e09caef1badfc189f42a609204fb38dea63bb1f1ae944 evidence/bounded_syn_activity.pcapng
695f2e51f86fc56e493e09caef1badfc189f42a609204fb38dea63bb1f1ae944 working/bounded_syn_activity_working.pcapng

```

Screenshot 10: Working copy created and SHA-256 hashes of evidence and working captures matched.

10. Part C - Identify SYN Indicators with tshark

PCAP=working/bounded_syn_activity_working.pcapng

cp --preserve=timestamps evidence/bounded_syn_activity.pcapng "\$PCAP"

sha256sum evidence/bounded_syn_activity.pcapng "\$PCAP" | tee reports/bounded_capture_hashes.txt

Initial SYNs only

tshark -r "\$PCAP" -Y 'tcp.flags.syn==1 && tcp.flags.ack==0' -T fields \

-e frame.number -e frame.time_epoch -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport -e tcp.seq \

| tee reports/initial_syns.tsv

SYN-ACK responses

tshark -r "\$PCAP" -Y 'tcp.flags.syn==1 && tcp.flags.ack==1' -T fields \

-e frame.number -e frame.time_epoch -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport -e tcp.ack \

| tee reports/syn_ack_responses.tsv

Resets and final ACK candidates

tshark -r "\$PCAP" -Y 'tcp.flags.reset==1 || (tcp.flags.ack==1 && tcp.len==0)' -T fields \

-e frame.number -e frame.time_epoch -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport -e tcp.flags \

| tee reports/ack_reset_candidates.tsv

```
(kali㉿kali)-[~/CIP-B103-Lab3]
$ tshark -r working/bounded_syn_activity_working.pcapng \
-Y 'tcp.flags.syn=1 && tcp.flags.ack=0' \
-T fields \
-e frame.number \
-e frame.time_epoch \
-e ip.src \
-e tcp.srcport \
-e ip.dst \
-e tcp.dstport \
-e tcp.seq \
| tee reports/initial_syns.tsv
1      1785080029.631886812      127.0.0.1      51631      127.0.0.1      80      0
4      1785080029.632391919      127.0.0.1      45323      127.0.0.1      80      0
7      1785080029.632815713      127.0.0.1      9145      127.0.0.1      80      0
10     1785080029.633296510      127.0.0.1      33581      127.0.0.1      80      0

(kali㉿kali)-[~/CIP-B103-Lab3]
$ tshark -r working/bounded_syn_activity_working.pcapng \
-Y 'tcp.flags.syn=1 && tcp.flags.ack=1' \
-T fields \
-e frame.number \
-e frame.time_epoch \
-e ip.src \
-e tcp.srcport \
-e ip.dst \

```

Screenshot 11: Initial SYN extraction and start of SYN-ACK response extraction.

```
-e tcp.dstport \
-e tcp.ack \
| tee reports/syn_ack_responses.tsv
2      1785080029.631907523      127.0.0.1      80      127.0.0.1      51631      1
5      1785080029.632400264      127.0.0.1      80      127.0.0.1      45323      1
8      1785080029.632823527      127.0.0.1      80      127.0.0.1      9145      1
11     1785080029.633305385      127.0.0.1      80      127.0.0.1      33581      1

(kali㉿kali)-[~/CIP-B103-Lab3]
$ tshark -r working/bounded_syn_activity_working.pcapng \
-Y 'tcp.flags.reset=1 || (tcp.flags.ack=1 && tcp.len=0)' \
-T fields \
-e frame.number \
-e frame.time_epoch \
-e ip.src \
-e tcp.srcport \
-e ip.dst \
-e tcp.dstport \
-e tcp.flags \
| tee reports/ack_reset_candidates.tsv
2      1785080029.631907523      127.0.0.1      80      127.0.0.1      51631      0x0012
3      1785080029.631913117      127.0.0.1      51631      127.0.0.1      80      0x0004
5      1785080029.632400264      127.0.0.1      80      127.0.0.1      45323      0x0012
6      1785080029.632403484      127.0.0.1      45323      127.0.0.1      80      0x0004
8      1785080029.632823527      127.0.0.1      80      127.0.0.1      9145      0x0012
9      1785080029.632826416      127.0.0.1      9145      127.0.0.1      80      0x0004
11     1785080029.633305385      127.0.0.1      80      127.0.0.1      33581      0x0012

```

Screenshot 12: SYN-ACK responses and RST/final-ACK candidate extraction results.

11. Part D - Quantify and Correlate the Activity

Counts by source and destination

```
tshark -r "$PCAP" -Y 'tcp.flags.syn==1 && tcp.flags.ack==0' -T fields \
-e ip.src -e ip.dst -e tcp.dstport | sort | uniq -c | sort -nr \
| tee reports/syn_counts_by_pair.txt
```

Unique client source ports

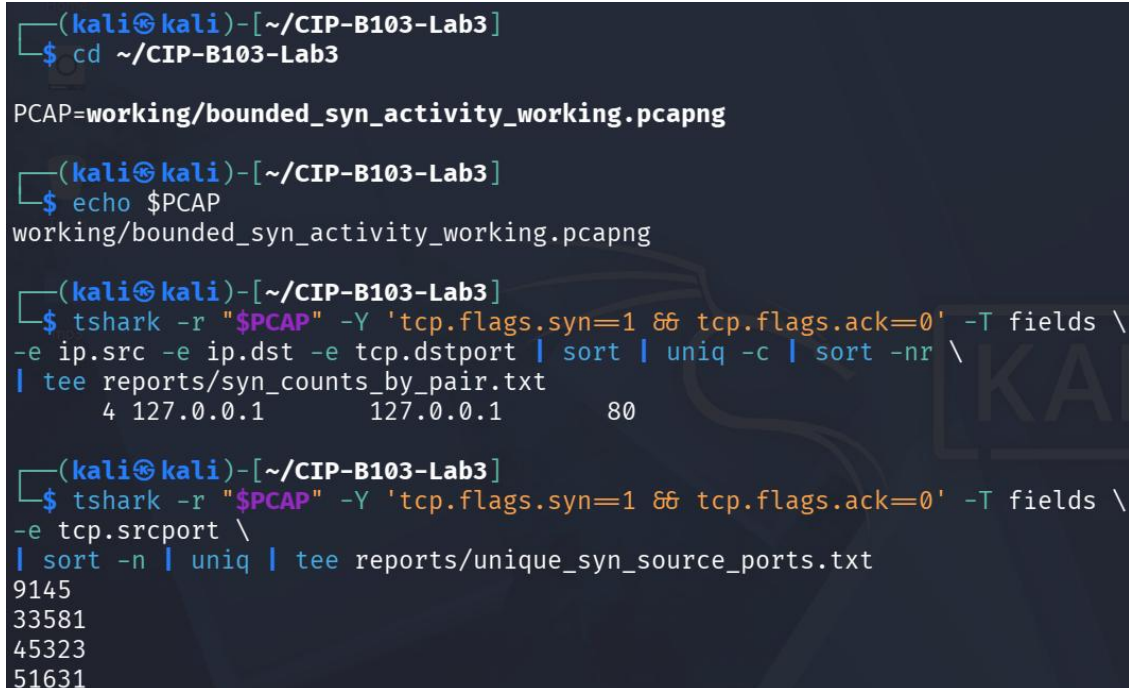
```
tshark -r "$PCAP" -Y 'tcp.flags.syn==1 && tcp.flags.ack==0' -T fields -e tcp.srcport \
| sort -n | uniq | tee reports/unique_syn_source_ports.txt
```

Expert information and TCP analysis flags

```
tshark -r "$PCAP" -q -z expert | tee reports/expert_info.txt
```

```
tshark -r "$PCAP" -Y 'tcp.analysis.retransmission || tcp.analysis.lost_segment ||
tcp.analysis.duplicate_ack' \
```

```
-T fields -e frame.number -e frame.time -e _ws.col.Info | tee reports/tcp_analysis_events.tsv
```



```
(kali㉿kali)-[~/CIP-B103-Lab3]
$ cd ~/CIP-B103-Lab3

PCAP=working/bounded_syn_activity_working.pcapng

(kali㉿kali)-[~/CIP-B103-Lab3]
$ echo $PCAP
working/bounded_syn_activity_working.pcapng

(kali㉿kali)-[~/CIP-B103-Lab3]
$ tshark -r "$PCAP" -Y 'tcp.flags.syn==1 && tcp.flags.ack==0' -T fields \
-e ip.src -e ip.dst -e tcp.dstport | sort | uniq -c | sort -nr \
| tee reports/syn_counts_by_pair.txt
      4 127.0.0.1          127.0.0.1          80

(kali㉿kali)-[~/CIP-B103-Lab3]
$ tshark -r "$PCAP" -Y 'tcp.flags.syn==1 && tcp.flags.ack==0' -T fields \
-e tcp.srcport \
| sort -n | uniq | tee reports/unique_syn_source_ports.txt
9145
33581
45323
51631
```

Screenshot 13: SYN counts by source/destination pair and unique source ports.


```

(kali㉿kali)-[~/CIP-B103-Lab3]
$ tshark -r "$PCAP" -q -z expert | tee reports/expert_info.txt

Warns (4)
=====
Frequency      Group      Protocol  Summary
      4      Sequence      TCP      Connection reset (RST)

Notes (4)
=====
Frequency      Group      Protocol  Summary
      4      Protocol      TCP      The SYN packet does not contain a SACK PERM option

Chats (8)
=====
Frequency      Group      Protocol  Summary
      4      Sequence      TCP      Connection establish request (SYN): server port 80
      4      Sequence      TCP      Connection establish acknowledge (SYN+ACK): server port 80

(kali㉿kali)-[~/CIP-B103-Lab3]
$ tshark -r "$PCAP" \
-Y 'tcp.analysis.retransmission || tcp.analysis.lost_segment || tcp.analysis.duplicate_ack' \
-T fields \
-e frame.number \
-e frame.time \
-e _ws.col.Info \

```

Screenshot 14: Expert information summary and TCP analysis events query.

12. Part E - Compare Normal and Suspicious Sessions

Indicator	Normal HTTP Session	Bounded SYN Activity	Forensic Meaning
Initial SYN count	1 (frame 1, source port 40198)	4 (frames 1, 4, 7, 10; ports 9145, 33581, 45323, 51631)	A single handshake attempt vs. four near-simultaneous SYNs from distinct ephemeral ports — the signature of scripted packet generation rather than normal browsing.
SYN-ACK count	1 (frame 2)	4 (frames 2, 5, 8, 11) — the server answered every SYN	Apache/the kernel responded 1:1 in both cases; SYN-ACK volume alone does not distinguish attack from bounded test — completion behaviour does.

Indicator	Normal HTTP Session	Bounded SYN Activity	Forensic Meaning
Completed handshakes	1 of 1 (confirmed by the successful HTTP GET/response in the curl output)	0 of 4 — each SYN-ACK was answered with an immediate client-side RST (frames 3, 6, 9 and the equivalent for stream 4) instead of a final ACK	Zero completed handshakes despite four SYN-ACKs is the core half-open indicator associated with SYN flood behaviour.
Unique client source ports	1 (40198)	4 (9145, 33581, 45323, 51631)	Multiple distinct ephemeral source ports appearing within milliseconds indicates scripted packet generation, not a single browser session.
HTTP request present?	Yes — GET / returned 10,703 bytes to curl	No — no application-layer traffic; only SYN/SYN-ACK/RST at the TCP layer	Absence of any application-layer exchange confirms the bounded activity never reached a usable connection.
RST packets	0	4 (one per stream, immediately following each SYN-ACK)	Client-generated RSTs after every SYN-ACK are consistent with a raw-socket SYN probe whose OS TCP stack rejects the unsolicited SYN-ACK.

Indicator	Normal HTTP Session	Bounded SYN Activity	Forensic Meaning
Observed duration	~20 s capture window; handshake and HTTP request completed within milliseconds of the curl call	~1.4 ms total spread across all 4 SYN/SYN- ACK/RST exchanges (frame timestamps 1785080029.6319– .6333)	Sub-millisecond spacing between four independent connection attempts is far faster than manual or normal client behaviour.

A real SYN flood is defined by scale, rate, persistence and service impact. Four training packets are not a denial-of-service event; they reproduce the packet pattern in a safe way so that analysts can learn the indicators.

```
(kali㉿kali)-[~/CIP-B103-Lab3]
$ tree ~/CIP-B103-Lab3
/home/kali/CIP-B103-Lab3
├── evidence
│   ├── bounded_syn_activity.pcapng
│   ├── mySYNFloodCapture.pcap
│   └── normal_http.pcapng
├── exported
├── reports
│   ├── ack_reset_candidates.tsv
│   ├── bounded_capture_hashes.txt
│   ├── bounded_syn_packets.tsv
│   ├── expert_info.txt
│   ├── initial_syns.tsv
│   ├── normal_handshake_flags.tsv
│   ├── syn_ack_responses.tsv
│   ├── syn_capture_sha256.txt
│   ├── syn_counts_by_pair.txt
│   ├── tcp_analysis_events.tsv
│   └── unique_syn_source_ports.txt
├── screenshots
├── scripts
│   └── syn_probe_lab.py
├── working
│   └── bounded_syn_activity_working.pcapng
└── 7 directories, 16 files
```

Screenshot 15: Final evidence, reports, scripts and working folder structure (tree).

13. Detection and Mitigation Recommendations

- Alert on elevated SYN-to-completed-handshake ratios per source and destination.

- Monitor SYN backlog exhaustion, retransmissions, connection-state tables and service latency.
- Use SYN cookies, sensible backlog/time-out tuning, upstream rate limiting and DDoS protection where appropriate.
- Retain full packet capture or flow telemetry during incidents and synchronize system clocks.
- Correlate packet evidence with web server, firewall, load balancer and operating-system logs.

Required Screenshots Checklist

- ☐ Apache service and TCP port 80.
- ☐ Normal handshake in the baseline capture.
- ☐ Bounded Scapy script showing target and count.
- ☐ Initial SYN packet list.
- ☐ SYN-ACK response list.
- ☐ Any ACK or RST behaviour.
- ☐ Counts by source/destination.
- ☐ Unique source ports.
- ☐ Normal-versus-suspicious comparison table.
- ☐ Evidence hashes.

Student Report Template

1. Executive summary.
2. Authority, scope and safety controls.
3. Evidence description and hashes.
4. Normal handshake baseline.
5. Suspicious-traffic indicators and timeline.
6. Quantitative analysis.
7. Assessment of whether the capture proves a flood, scan or bounded simulation.
8. Detection and mitigation recommendations.
9. Appendix of commands and screenshots.

Submission Requirements

- One PDF report named CIP-B103-Lab3_RegNo_FullName.pdf.
- The original or generated PCAP/PCAPNG evidence file and its SHA-256 hash text file.

- A reports folder containing exported CSV/TXT command output and any recovered objects.
- Screenshots must be readable, numbered and referenced in the report narrative.
- Compress the report and evidence outputs into one ZIP named with your registration number and lab number.
- Do not submit passwords or decoded credentials in public discussion channels; include them only in the protected assessment submission where required.

Marking Rubric (100 Marks)

Assessment Area	Marks	What the Instructor Will Check
Safety and evidence handling	15	Bounded loopback/host-only scope, hashes and clean stop.
Baseline analysis	15	Correct normal handshake identification.
tshark filtering	20	Accurate extraction of SYN, SYN-ACK, ACK/RST and fields.
Quantitative findings	20	Counts, ports, timing and correlation are correct.
Forensic interpretation	20	Clearly distinguishes packet pattern from proof of full DoS impact.
Report quality	10	Professional narrative and screenshots.

Instructor Quick Answer Guide

Checkpoint	Expected Observation
Initial SYN filter	<code>tcp.flags.syn==1 && tcp.flags.ack==0.</code>
SYN-ACK filter	<code>tcp.flags.syn==1 && tcp.flags.ack==1.</code>

Checkpoint	Expected Observation
Normal handshake	Final ACK follows SYN-ACK and application data may follow.
Suspicious pattern	Many initial SYNs with few completed handshakes.
Training limitation	Four packets demonstrate indicators but do not prove service denial.
Strong conclusion	Requires rate, duration, system impact and corroborating logs.

Command Reference Summary

Command	Purpose
tshark -r file -Y FILTER	Analyze a saved capture.
tcp.flags.syn	Filter on the SYN flag.
tcp.flags.ack	Filter on the ACK flag.
tcp.stream	Conversation identifier.
tshark -q -z expert	Show expert analysis.
sort uniq -c	Count repeated field combinations.

Academic Integrity and Authorization Statement

Student Declaration

I confirm that I completed this lab in an authorized environment, preserved the supplied evidence, did not target any third-party system, and accurately documented my own commands, observations and conclusions.